Happy Pranavs
Dull Kim, Vishwin Siddi

Github
https://github.com/pranavsachoo/pranavs

Goals
- Original Goal:
To seek spotify's API and use BS4 with turtlefind.com to find music in movies / shows, and determine if the mood of the music used matches the genre of the movie / show
- Revised Goal:
To gather top 3 movies and videogames since 2000 to 2024, process and compare reviews, genre, and ratings, etc. We wanted to see if there would be any similarities or differences between these two industries.

Problems Faced
- The biggest problem was that Spotify didn't have python support and the api endpoints we wanted to use were deprecated in Spotify too.
- So, we switched our goals and decided to compare movies and books. We wanted to see if there were any cross-to-difference when movies were converted to books, and books converted to movies.
- We couldn't do this one either because none of the books APIs actually give you ratings / reviews of the book. This is why you'll see some the names and code in the first few controls that are for the book API.
Hence, we finally decided on comparing each year's top 5 video games and movies on ratings, vote counts, genre, etc.

Calculation / Visual 1

Code:

```
# Imports
import sqlite3
import matplotlib.pyplot as plt
import numpy as np

# Connecting db
conn = sqlite3.connect("pranavs.db")
c = conn.cursor()

# Getting movie bits per year (rating >= 8)
c.execute("""
    SELECT year, SUM(CASE WHEN rating >= 8 THEN 1 ELSE 0 END)
    FROM movies
    GROUP BY year
    ORDER BY year
    LIMIT 5
""")
movie_bits = c.fetchall()

# Getting game bits per year (rating >= 8 on 0-5)
c.execute("""
    SELECT year, SUM(CASE WHEN rating >= 8 THEN 1 ELSE 0 END)
    FROM games
    GROUP BY year
    ORDER BY year
    LIMIT 5
""")
game_bits = c.fetchall()

# Unpacking
years = [y for y, _ in movie_bits]
movie_counts = [b for _, b in movie_bits]
game_counts = [b for _, b in game_bits]

# Plot
x = np.arange(len(years))
width = 0.35
plt.bar(x - width/2, movie_counts, width, label='Movie')
plt.bar(x + width/2, game_counts, width, label='Game')
plt.xlabel('Year')
plt.ylabel('Genre')
plt.title('Comparison Across Last Five Years')
plt.xticks(x, years)
plt.legend()
plt.show()
```

Calculation / Visual 2

Code:

```
# Imports
import sqlite3
import matplotlib.pyplot as plt

# Connecting db
conn = sqlite3.connect("pranavs.db")
c = conn.cursor()

# Getting movie hits per year (rating >= 8)
c.execute("""
    SELECT year, SUM(CASE WHEN rating >= 8 THEN 1 ELSE 0 END)
    FROM movies
    GROUP BY year
    ORDER BY year
    LIMIT 5
""")
movie_hits = c.fetchall()

# Getting game hits per year (rating >= 8 on 0-5)
c.execute("""
    SELECT year, SUM(CASE WHEN rating >= 8 THEN 1 ELSE 0 END)
    FROM games
    GROUP BY year
    ORDER BY year
    LIMIT 5
""")
game_hits = c.fetchall()

conn.close()

# Getting movie data from tables into lists
movie_years = []
movie_hits = []
for row in movies:
    movie_years.append(row[0])
    movie_hits.append(row[1])

# Getting game data from tables into lists
game_years = []
game_hits = []
for row in games:
    game_years.append(row[0])
    game_hits.append(row[1])

# Plot
plt.figure(figsize=(10, 5))
plt.plot(movie_years, movie_hits, label='Movies', alpha=0.5, color='purple')
plt.plot(game_years, game_hits, label='Games', alpha=0.5, color='red')
plt.xlabel('Year')
plt.ylabel('Hit Count')
plt.title('Movie and Game Hits per Year (A lot is when rating is > 80%)')
plt.legend()
plt.show()
```

Calculation / Visual 3

Code:

```
# Imports
import sqlite3
import matplotlib.pyplot as plt
import numpy as np

# Connecting db
conn = sqlite3.connect("pranavs.db")
c = conn.cursor()

# Getting all movies ratings, and vote counts
c.execute("SELECT rating, votes FROM movies")
movies = c.fetchall()

# Getting all games ratings, and vote counts
c.execute("SELECT rating, votes FROM games")
games = c.fetchall()

conn.close()

# Building movie data from tables into lists.
movie_ratings = []
movie_votes = []
for row in movies:
    rating = row[0]
    votes = row[1]
    movie_ratings.append(rating)
    movie_votes.append(votes)

# Building game data from tables into lists.
game_ratings = []
game_votes = []
for row in games:
    rating = row[0]
    votes = row[1]
    game_ratings.append(rating)
    game_votes.append(votes)

# Scale votes to 0-100 range (relative to each industry's max)
movie_max = max(v for v in movie_votes if v is not None)
game_max = max(v for v in game_votes if v is not None)
movie_votes_scaled = [(v / movie_max) * 100 if v is not None else None for v in movie_votes]
game_votes_scaled = [(v / game_max) * 100 if v is not None else None for v in game_votes]

# Plot
plt.scatter(movie_ratings, movie_votes_scaled, alpha=0.5, color='purple')
plt.scatter(game_ratings, game_votes_scaled, alpha=0.5, color='red')
plt.xlabel('Average Rating')
plt.ylabel('Popularity')

# Legend
plt.scatter(movie_ratings, movie_votes_scaled, alpha=0.5, color='red')
plt.scatter(game_ratings, game_votes_scaled, alpha=0.5, color='purple')
plt.xlabel('Average Rating')
plt.ylabel('Popularity')

# Displaying
plt.suptitle('Relative Popularity vs. Ratings for Movies and Games')
plt.show()
```

Calculation / Visual 4

Code:

```
# Imports
import sqlite3
import matplotlib.pyplot as plt
import numpy as np

# Connecting db
conn = sqlite3.connect("pranavs.db")
c = conn.cursor()

# Getting all movies ratings, and vote counts.
c.execute("SELECT rating, votes FROM movies")
movies = c.fetchall()

# Getting all games ratings, and vote counts.
c.execute("SELECT rating, votes FROM games")
games = c.fetchall()

conn.close()

# Building movie data from tables into lists.
movie_ratings = []
movie_votes = []
for row in movies:
    rating = row[0]
    votes = row[1]
    movie_ratings.append(rating)
    movie_votes.append(votes)

# Building game data from tables into lists.
game_ratings = []
game_votes = []
for row in games:
    rating = row[0]
    votes = row[1]
    game_ratings.append(rating)
    game_votes.append(votes)

# Scale votes to 0-100 range (relative to each industry's max)
movie_max = max(v for v in movie_votes if v is not None)
game_max = max(v for v in game_votes if v is not None)
movie_votes_scaled = [(v / movie_max) * 100 if v is not None else None for v in movie_votes]
game_votes_scaled = [(v / game_max) * 100 if v is not None else None for v in game_votes]

# Plot
plt.scatter(movie_ratings, movie_votes_scaled, alpha=0.5, color='purple')
plt.scatter(game_ratings, game_votes_scaled, alpha=0.5, color='red')
plt.xlabel('Average Rating')
plt.ylabel('Popularity')

# Legend
plt.scatter(movie_ratings, movie_votes_scaled, alpha=0.5, color='red')
plt.scatter(game_ratings, game_votes_scaled, alpha=0.5, color='purple')
plt.xlabel('Average Rating')
plt.ylabel('Popularity')

# Displaying
plt.suptitle('Relative Popularity vs. Ratings for Movies and Games')
plt.show()
```

Calculation / Visual 5

Code:

```
# Imports
import sqlite3
import matplotlib.pyplot as plt
import numpy as np

# Connecting db
conn = sqlite3.connect("pranavs.db")
c = conn.cursor()

# Getting all movies ratings, and vote counts.
c.execute("SELECT rating, votes FROM movies")
movies = c.fetchall()

# Getting all games ratings, and vote counts.
c.execute("SELECT rating, votes FROM games")
games = c.fetchall()

conn.close()

# Building movie data from tables into lists.
movie_ratings = []
movie_votes = []
for row in movies:
    rating = row[0]
    votes = row[1]
    movie_ratings.append(rating)
    movie_votes.append(votes)

# Building game data from tables into lists.
game_ratings = []
game_votes = []
for row in games:
    rating = row[0]
    votes = row[1]
    game_ratings.append(rating)
    game_votes.append(votes)

# Scale votes to 0-100 range (relative to each industry's max)
movie_max = max(v for v in movie_votes if v is not None)
game_max = max(v for v in game_votes if v is not None)
movie_votes_scaled = [(v / movie_max) * 100 if v is not None else None for v in movie_votes]
game_votes_scaled = [(v / game_max) * 100 if v is not None else None for v in game_votes]

# Plot
plt.scatter(movie_ratings, movie_votes_scaled, alpha=0.5, color='purple')
plt.scatter(game_ratings, game_votes_scaled, alpha=0.5, color='red')
plt.xlabel('Average Rating')
plt.ylabel('Popularity')

# Legend
plt.scatter(movie_ratings, movie_votes_scaled, alpha=0.5, color='red')
plt.scatter(game_ratings, game_votes_scaled, alpha=0.5, color='purple')
plt.xlabel('Average Rating')
plt.ylabel('Popularity')

# Displaying
plt.suptitle('Relative Popularity vs. Ratings for Movies and Games')
plt.show()
```

Instructions to run code:
- Creating environment variables:
  - At the time of writing this report, I'm not sure if we're including the keys in the file or not, so I will include instructions for them:
  - Create a .env file in the root folder (the one main.py is in)
  - Set TMDB_KEY = "<key>"
  - Set RAWG_KEY = "<key>"
  - You can get TMDB and RAWG keys using these sites:
    - TMDB https://developer.themoviedb.org/docs/getting-started
    - RAWG https://rawg.io/apidocs
- Gathering the data:
  - The code runs in this format:
    Define Row Cleaner
    Fetching
    Insert Into Cursor
    Loop
  - So, run each python file get 125 rows in both tables.
  - Run the file until you see the message: "Can't run more, limit year 2024 reached."
  - pranavs.db should now be created and visible in the folder.
- Visualizing the data:
  - Run the visual .py files present in the root folder, for each respective visual.

Function Diagram

Resources Table

Date	Issue Desc	Location	Result (did it solve issue?)
12 Nov	Finding APIs for music in movies/shows	https://github.com/public-apis/public-apis	Yes - Found Spotify, initially planned
28 Nov	Spotify Python support.	https://spotipy.readthedocs.io/	No - API endpoints deprecated, switched goals.
28 Nov	Google APIs lacking ratings/ reviews.	https://developers.google.com/custom-search/docs/tutorial/introduction?hl=en#prerequisites_to_use#check_api_keys https://www.reddit.com/r/AskPythonDev/comments/1ftqwq/ https://www.reddit.com/r/androidstudio/comments/7ocxdf/review_scores_and/	No - No ratings data, abandoned books/movies.
2 Dec	Finding movie/game comparison APIs.	https://github.com/public-apis/public-apis https://zuplo.com/learning-center/top-movie-api-vs-omdb-vs-tmdb	Yes - Selected TMDB + RAWG for final project.
7 Dec	API key setup.	https://developer.themoviedb.org/docs/getting-started https://rawg.io/apidocs	Yes - Obtained both keys.
7 Dec	Environment variables vs.txt file for API keys.	https://stackoverflow.com/questions/30700348/python-dotenv	Yes - Created .env and started using dotenv library.